

BASELINE · SECURITY REVIEW

Baseline Round 2.

A security review of Round 2.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Baseline	Round 2	Guardian	October 20 – December 23, 2025	December 30, 2025

§ 00

Contents.

74 pages — overview, confidence ranking, scope, 47 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 45 findings	P. 10
	Remediation Review · 2 findings	P. 70
§ 05	Disclaimer	P. 73
§ 06	About Guardian	P. 74

§ 01

Overview.

Guardian reviewed Round 2 across 2 review rounds. The final report includes 4 Critical, 14 High, 8 Medium, 13 Low, 8 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	0xBaseline/mercury
CLIENT	METHODOLOGY
Baseline	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
Round 2	db526b87e5b9
PREPARED BY	REMEDATION REVIEW COMMIT
Guardian	7cd5a5d3c159
AUDIT WINDOW	
October 20 – December 23, 2025	
FINAL REPORT	
December 30, 2025	

1.1 Confidence Ranking

1 / 5

Very Low Confidence

Guardian assigns a Confidence Ranking of 1 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.



1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
● Critical	4	0	1	1	2
● High	14	0	4	0	10
● Medium	8	0	6	0	2
● Low	13	0	2	0	11
● Informational	8	0	6	0	2
All severities	47	0	19	1	27

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence ASSIGNED	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Round 2 in `0xBaSeLine/mercury`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Leverage Mistakes Debt As Yield	Logical Error	CRITICAL	PARTIALLY RESOLVED	p. 10
C-02	Incorrect Rounding Allows Price Collapse	Rounding	CRITICAL	RESOLVED	p. 11
C-03	Arbitrage Guards Bypassed	Gaming	CRITICAL	ACKNOWLEDGED	p. 13
C-04	Withdrawing collateral without repaying debt	Logical Error	CRITICAL	RESOLVED	p. 15
H-01	pnOffset Uses Incorrect Decimals	Logical Error	HIGH	ACKNOWLEDGED	p. 17
H-02	Missing Harvest DoS's Buys	DoS	HIGH	RESOLVED	p. 18
H-03	Exit Perturbs Pool Reserves By Missing Sweep	Logical Error	HIGH	RESOLVED	p. 19
H-04	Sweep DoS	DoS	HIGH	RESOLVED	p. 20
H-05	Xmin Slippage Can Be Bypassed	Logical Error	HIGH	ACKNOWLEDGED	p. 21
H-06	defaultSelf() issues	Logical Error	HIGH	RESOLVED	p. 22
H-07	Yield can be manipulated	Gaming	HIGH	RESOLVED	p. 24
H-08	Vault Rounding Perturbs Accounting	Rounding	HIGH	RESOLVED	p. 26
H-09	Swap reentrancy enables yield manipulation	Reentrancy	HIGH	RESOLVED	p. 28
H-10	Leverage call is vulnerable to MEV sandwiches	MEV	HIGH	ACKNOWLEDGED	p. 30

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
H-11	Inflation caused by claimed credits	Logical Error	HIGH	RESOLVED	p. 31
H-12	Slippage Affected By Circulating Supply	Gaming	HIGH	RESOLVED	p. 33
H-13	Arb Guard Does Not Effectively Protect Execution	Gaming	HIGH	ACKNOWLEDGED	p. 36
M-01	Unused Benchmark Offset	Logical Error	MEDIUM	ACKNOWLEDGED	p. 37
M-02	Incorrect PnL offset for short positions	Logical Error	MEDIUM	ACKNOWLEDGED	p. 38
M-03	Vault interactions don't check its limits	Compatibility	MEDIUM	ACKNOWLEDGED	p. 39
M-04	Pool creator can steal reserves from the pool	Reentrancy	MEDIUM	RESOLVED	p. 40
M-05	Yield miscalculation because of deLeverage()	Logical Error	MEDIUM	RESOLVED	p. 42
M-06	Execution price doesn't account for fees on sell	Logical Error	MEDIUM	ACKNOWLEDGED	p. 43
M-07	Bid inverse pricing asymmetry	Logical Error	MEDIUM	ACKNOWLEDGED	p. 44
M-08	Risk of insolvency due to debt	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 46
L-01	Unnecessary Price Comparison	Gas Optimization	LOW	RESOLVED	p. 47
L-02	AdminTransferred event is emitted twice	Events	LOW	RESOLVED	p. 48
L-03	Initiating ownership transfer should emit event	Events	LOW	RESOLVED	p. 50
L-04	Component doesn't properly support ERC165	Compatibility	LOW	RESOLVED	p. 51
L-05	claimPoolFees Does Not Sweep	Unexpected Behavior	LOW	RESOLVED	p. 52
L-06	getBufferConvexity() is not accessible	DoS	LOW	RESOLVED	p. 53

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-07	Rounding Asymmetry Causes getMaxBorrow Revert	DoS	LOW	RESOLVED	p. 54
L-08	Pool can be misconfigured	Validation	LOW	RESOLVED	p. 56
L-09	Possible division by 0	Math	LOW	ACKNOWLEDGED	p. 57
L-10	Leverage rounds in users favor	Rounding	LOW	RESOLVED	p. 58
L-11	Bid inverse calculates some values in user favor	Rounding	LOW	RESOLVED	p. 59
L-12	Wrong benchmarks due to dynamic total supply	Logical Error	LOW	RESOLVED	p. 61
I-01	Labels cause truncation of longer contract names	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 62
I-02	Native token is not supported	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 63
I-03	Swap fails if hookData.length is in the range [1;32)	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 64
I-04	solveBlvForConvexity() is a misleading name	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 65
I-05	Incorrect PnL comment	Best Practices	INFORMATIONAL	RESOLVED	p. 66
I-06	borrowWithNative() should be renamed	Best Practices	INFORMATIONAL	RESOLVED	p. 67
I-07	sync() frequency impacts the error correction	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 68
I-08	Console logs should be removed	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 69
REMEDATION REVIEW					
H-01	Fees Perturb Harvestable Yield	Logical Error	HIGH	RESOLVED	p. 70
L-01	No Fee Refund	Warning	LOW	ACKNOWLEDGED	p. 72

CRITICAL C-01

PARTIALLY RESOLVED

Leverage Mistakes Debt As Yield

CATEGORY	Logical Error
LOCATION	BCredit.sol
REVIEW	Main Review

DESCRIPTION

When a pool is allocated to a vault, the harvestable yield is calculated as the amount in excess of a tracked watermark. In the `getHarvestableYield` function this watermark is defined as `pool.totalReserves - credit.totalDebt + unclaimedFees` and is compared to the value of the systems shares of the vault token.

However in the leverage function, the `totalDebt` and pool `totalReserves` are updated before the vault shares are redeemed and deducted from the `pool.shareBalance`. And the `BStaking.deposit` function, which updates the harvestable yield is called after the `totalDebt` update and before the `shareBalance` deduction. As a result, the `totalDebt` that was just incurred appears like an immediate yield, since the `shareBalance` does not decrease, but the watermark does.

This inflates the yield generated by the vault significantly when a leverage action occurs where the debt borrowed is greater than the collateral purchase cost, in other words, when user's receive reserves out of the leverage which would be removed from the vault.

Furthermore, when the `reserveDelta` would be positive during the leverage flow, e.g. when the user must pay net reserves in. The yield can be accidentally decreased because the current worth of the vault is compared to the watermark before depositing the new reserves from the user to receive additional vault shares.

RECOMMENDATION

Perform the `BStaking` deposit and lock on behalf of the user at the end of the leverage flow, after the vault has been deposited into or withdrawn from. This way the yield calculation reflects the final current state of the system.

RESOLUTION

Status: Partially resolved.

<https://github.com/0xBaseline/mercury/blob/ec7116192eec14270a5a5bd108f30f305f0a7d86/src/components/BCredit.sol#L227-L229>

CRITICAL C-02

RESOLVED

Incorrect Rounding Allows Price Collapse

CATEGORY	Rounding
LOCATION	BSwap.sol
REVIEW	Main Review

DESCRIPTION

In the `quoteReservesForTokensOut` function the protocol is careful to round against the user and in the favor of the AMM to maximize the amount of reserves collected from the user.

However when denormalizing the amount of reserves to actually collect from the user the `denormalizeWad` function is used which has no rounding logic and instead always rounds down. This means that for pools where the reserve token has less than 18 decimals a non-trivial amount of reserves were computed as having come from the user in the quote logic, but that are not actually supplied by the user.

This is especially bad because the calculation of `pAvg` will use this dust size trade execution as the average execution price to determine the safe price for a sell action. The safe price may then be far lower than the actual current pool price, and subsequently becomes the new pool price after the sell execution. Now users are able to buy BTokens using the full liquidity of the pool at this much lower manipulated price.

Consider the following case:

- BTokens have 18 decimals, reserves have 6 decimals
- Trader A executes a swap where they buy $0.01e18$ BTokens for a normalized $1.9e12$ reserves in
- The $1.9e12$ reserves are denormalized to 1 wei of reserves
- The quoted execution rate was $1.9e12 / 0.01e18 = 0.00019$
- The actual execution rate was $1e12 / 0.01e18$ BTokens reserves = 0.0001
- The actual rate was almost 50% cheaper than the quoted execution rate, this now becomes the new regime's average price
- The attacker triggers a small sell, the `pAvg` calculated for the safe bid price calculation uses this average price which was only experienced for dust amounts, this now becomes the active price of the pool
- Anyone can now buy BTokens at roughly half off because the pool price has been assigned to this much lower "safe price" that was assigned

C-02 Incorrect Rounding Allows Price Collapse

CONTINUED

— RECOMMENDATION

Introduce directional rounding for normalization and denormalization and be sure to round against the user and in the favor of the protocol in all normalization and denormalization related activities.

PROOF OF CONCEPT https://github.com/GuardianOrg/mercury-baselineamm-fuzz/blob/202510_Mercury_Round2_Fuzzing/test/unit/BSwapRoundingPoC.t.sol

— RESOLUTION

Status: Resolved.

<https://github.com/0xBaseline/mercury/blob/00543dceaec00b341e6df0facd5507e04780a613/src/components/BSwap.sol#L314>

CRITICAL C-03

ACKNOWLEDGED

Arbitrage Guards Bypassed

CATEGORY Gaming
LOCATION BSwap.sol
REVIEW Main Review

DESCRIPTION

Arbitrage guards such as the `snapshotCirculating` and `snapshotReserves` have been introduced to avoid allowing any net positive gain swaps for malicious actors.

The arbitrage guard is enforced in the `computeSellTokens` function when the `maker.accumulatedDelta` is negative, indicating a current buy regime.

However in the `_updateArbGuard` function the `accumulatedDelta` is reset to the new `_tokenDelta` immediately whenever an action is a reversal of the current trend:

```
bool isReversal = (accumulatedDelta > 0 && _tokenDelta < 0) ||  
(accumulatedDelta < 0 && _tokenDelta > 0);
```

However this means that a malicious actor can easily avoid the arb guard after buying tokens buy initiating a small sell to reset the `_maker.accumulatedDelta` value to a positive one, thus avoiding the `getSafePriceBid` constraint in the `computeSellTokens` or `computeSellReserves` functions.

This allows for net gain arbs as shown in the attached PoC.

RECOMMENDATION

Consider updating the `isReversal` criteria to only reset the snapshots when the net volume switches sides from net buy to net sell or vice versa.

```
int256 nextAccumulatedDelta = accumulatedDelta + _tokenDelta;  
// Boolean to check if trade is adding to the trend or reversing it  
bool isReversal = (accumulatedDelta > 0 && nextAccumulatedDelta < 0) ||  
(accumulatedDelta < 0 && nextAccumulatedDelta > 0);
```

And always updating the `accumulatedDelta` to simply reflect whether the pool is in a net buy or net sell.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-baselineamm-fuzz/blob/5abae8aa98c31248cfdbf9cddbfbf8129a88ec1a4/test/fuzzing/FoundryPlayground.sol#L14>

C-03 Arbitrage Guards Bypassed

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

CRITICAL C-04

RESOLVED

Withdrawing collateral without repaying debt

CATEGORY Logical Error
LOCATION BCredit.sol#L149
REVIEW Main Review

DESCRIPTION

Upon creating a pool, `initialCollateral` and `initialDebt` are assigned to the `Relay` address in the credit system. These amounts are to be claimed by users provided in the merkle root configuration. As users claim, both the collateral and the debt is transferred to them from the Relay credit account and the amount is deposited to their staking account.

```
BStaking(address(this)).deposit(_bToken, msg.sender, _collateral);  
// update the proxy credit account  
credit.accounts[address(this)].collateral -= _collateral;  
credit.accounts[address(this)].debt -= _debt;  
// update the user's credit account  
credit.accounts[msg.sender].collateral += _collateral;  
credit.accounts[msg.sender].debt += _debt;
```

They should repay their debt afterwards and unlock their collateral. However, the collateral was never locked to begin with. This lets all of the users eligible for a claim withdraw their collateral via `BStake.withdraw()` and leave the debt unpaid. Furthermore, honest users which try to repay their debt first will have their transactions reverted because `repay()` will try to unlock collateral that has never been locked.

RECOMMENDATION

Lock the user collateral after depositing it to the stake in `BCredit.claimCredit()`:

```
BStaking(address(this)).deposit(_bToken, msg.sender, _collateral);  
+ State.staking(_bToken).lockCollateral(msg.sender, _collateral);
```

PROOF OF CONCEPT <https://gist.github.com/ZdravkoHr/826719c18bafcb3d4e985ae529df9994>

RESOLUTION

Status: Resolved.

C-04 Withdrawing collateral without repaying debt

CONTINUED

<https://github.com/0xBaseline/mercury/blob/ec7116192eec14270a5a5bd108f30f305f0a7d86/src/components/BCredit.sol#L150-L151>

HIGH **H-01**

ACKNOWLEDGED

pnlOffset Uses Incorrect Decimals

CATEGORY Logical Error
LOCATION BSwap.sol
REVIEW Main Review

DESCRIPTION

In the `_updateAsk` function the `inventoryOffset` and `pnlOffset` serve as two distinct caps on the `offsetSupply`. The `inventoryOffset` is derived from the `_params.poolReserves` and `_params.poolTokens` values which have been normalized to 18 decimals.

However the `pnlOffset` relies on the `pool.totalBTokens` value which has not been normalized to 18 decimals. Thus the `pnlOffset` is made inconsequential when the BToken has less than 18 decimals, and perturbs the rest of the `offsetSupply` capping when the BToken has greater than 18 decimals.

RECOMMENDATION

Use the normalized `params.poolTokens` instead of the unnormalized `pool.totalBTokens` to compute the `pnlOffset` value.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-team1-1760986550486/pull/1>

RESOLUTION

Status: Acknowledged.

HIGH **H-02**

RESOLVED

Missing Harvest DoS's Buys

CATEGORY	DoS
LOCATION	VaultLib.sol: 151
REVIEW	Main Review

DESCRIPTION

In the `exitVault` function the `pool.totalReserves` value is re-assigned to account for the redeemed amount from the vault after pulling all funds less the unclaimed fees value.

However, the unclaimed fees has not been increased by the pending yield amount if any has been accrued, as a result the `totalReserves` can unexpectedly increase as a result of exiting the vault.

This unexpected increase, where the yield incorrectly goes straight to the pool reserves rather than the fee collectors and stakers, leads to a DoS on token buys in the `getSafePriceAsk` function because the `poolReserves` amount is unexpectedly larger than the `snapshotReserves` amount, leading to an underflow DoS.

It should never be the case that in a regime of selling the `snapshotReserves` are less than the `poolReserves`, and this lacking harvest violates that invariant.

RECOMMENDATION

Harvest the pending yield before pulling out of the vault in the `exitVault` function.

RESOLUTION

Status: Resolved.

<https://github.com/0xBaseline/mercury/blob/ec7116192eec14270a5a5bd108f30f305f0a7d86/src/libraries/VaultLib.sol#L178>

HIGH **H-03**

RESOLVED

Exit Perturbs Pool Reserves By Missing Sweep

CATEGORY Logical Error
LOCATION VaultLib.sol
REVIEW Main Review

DESCRIPTION

In the `exitVault` function there is no logic to sweep the tokens that may be sitting within the `poolManager` contract instead of the vault. This perturbs the `_pool.totalReserves` assignment that occurs in the `exitVault` function because these token amounts are not included in the redeemed value, thus making the pool appear as if it has much less reserves than it actually does.

RECOMMENDATION

Enforce a sweep at the very beginning of the `exitVault` function.

RESOLUTION

Status: Resolved.

<https://github.com/0xBaseline/mercury/blob/ec7116192eec14270a5a5bd108f30f305f0a7d86/src/libraries/VaultLib.sol#L191>

HIGH **H-04**

RESOLVED

Sweep DoS

CATEGORY	DoS
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

The sweep action may revert in cases where dust is held in the poolManager contract that results in 0 shares being awarded to the pool upon depositing into the underlying vault.

Most vault implementations simply revert when zero shares are awarded, thus blocking actions like swaps and vault updates when there are dust amounts to be claimed from the poolManager.

A malicious actor could intentionally perform tiny swaps through the router in order to continuously DoS swaps and vault updates for any pool that has both a Uniswap hook and a vault configured.

— RECOMMENDATION

If the sweep would result in zero vault shares being awarded to the system, consider leaving the dust tokens in the pool manager.

— RESOLUTION

Status: Resolved.

<https://github.com/0xBaseline/mercury/blob/ec7116192eec14270a5a5bd108f30f305f0a7d86/src/libraries/SweepLib.sol#L99>

HIGH **H-05**

ACKNOWLEDGED

Xmin Slippage Can Be Bypassed

CATEGORY Logical Error
LOCATION BSwap.sol
REVIEW Main Review

DESCRIPTION

In the BToken buy flow, in the `_recordSwap` function the `_updateBenchmark` function will reset the `offsetSupply` to zero when the book value increases and the poolTokens decreases.

This will often occur on a BToken buy, since the book value is represented as the reserves in the pool (increases with a BToken buy) divided by the circulating supply (increases with a BToken buy) and thus any buy that occurs above book price increases the book price. Furthermore, the poolTokens will always decrease on a BToken buy, qualifying it for this case.

When the `_bTokenDelta` is negative, as is the case for BToken buys, the `_updateBid` function is used which does not re-assign the `offsetSupply`. Therefore the buy leaves the `offsetSupply` at 0, no matter how large the offset was prior.

This means that buyers can significantly improve the execution for their own buys and all following buys by first triggering this case before performing subsequent buys. This way the natural AMM dynamics of the Xmin value are effectively bypassed at the expense of the protocol.

RECOMMENDATION

Do not re-assign the `offsetSupply` to zero in the `_updateBenchmark` function and consider computing and assigning what it ought to be in the `_updateBid` function.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-baselineamm-fuzz/blob/05bbb204ad21fcc8eecd5f7dbf75cfb1dabc5c5/test/unit/BSwap.t.sol#L213>

RESOLUTION

Status: Acknowledged.

<https://github.com/0xBaseline/mercury/blob/00543dceaec00b341e6df0facd5507e04780a613/src/components/BSwap.sol#L613>

HIGH **H-06**

RESOLVED

defaultSelf() issues

CATEGORY	Logical Error
LOCATION	BCredit.sol#L162-176
REVIEW	Main Review

DESCRIPTION

`BCredit.defaultSelf()` is a function that anyone to exit their credit position - losing their collateral and having their debt forgiven. There are a few issues with the function.

First, its selector is not exposed in the `ROUTES()` function so at the moment it cannot be called on the `Relay`.

Second, it executes `credit.totalDebt -= debt`, which means `debt` amount of the reserves are leaving the system forever and `collateral` amount of `bToken` are incoming, therefore `pool.totalReserves` should be decreased and `pool.totalBTokens` increased. This is not happening in the current implementation which will break the tokens accounting in the system as more and more assets are being moved. One example of where it creates a problem is when the harvestable yield is calculated.

```
uint256 assetsWithYield = getAllocatedReserves(pool);  
uint256 holdings = pool.totalReserves - State.credit(_bToken).totalDebt +  
FeeLib.getUnclaimedFees(_bToken);
```

The `totalDebt` is subtracted from `totalReserves`, so when the debt is cleared, `holdings` will experience an increase in its value, while `assetsWithYield` stays the same. In contrast, if `repay()` happened instead of `defaultSelf()`, both `holdings` and `assetsWithYield` would have increased.

Third, the user collateral is reduced to 0 and is not unlocked. While this ensures they cannot withdraw it, their staking position remains active and keeps accumulating yield, resulting in permanently inaccessible rewards at the expense of other participants in the system.

Fourth, in practice, this function allows users to perform `sellTokens` bypassing `BSwap.sellTokens()`, but at exactly `BVL` price, which means they lose the premium. This behavior is dangerous because it performs swap, but doesn't update the curve, nor it applies any other restrictions.

H-06 defaultSelf() issues

CONTINUED

RECOMMENDATION

If the function is not needed, consider removing it. Otherwise, fix the issues from the report and consider adding the `permissioned` modifier to reduce the risk of having unauthorized swaps.

RESOLUTION

Status: Resolved.

<https://github.com/0xBaseline/mercury/blob/ec7116192eec14270a5a5bd108f30f305f0a7d86/src/components/BCredit.sol#L174-L184>

HIGH **H-07**

RESOLVED

Yield can be manipulated

CATEGORY Gaming
LOCATION VaultLib.sol#L56-59
REVIEW Main Review

DESCRIPTION

`VaultLib.getHarvestableYield()` subtracts the `holdings` from the `assetsWithYield` to calculate the harvestable yield.

```
uint256 assetsWithYield = getAllocatedReserves(pool);  
uint256 holdings = pool.totalReserves - State.credit(_bToken).totalDebt +  
FeeLib.getUnclaimedFees(_bToken);  
return int256(assetsWithYield) - int256(holdings) - 1;
```

The `assetsWithYield` value stores the notional deposited value to the vault plus any yield earned.

Direct buys through the UniswapV4 pool update `pool.totalReserves`, but because the assets are not yet in the system, they are not deposited to the vault. This causes unilateral increase in the above expression and results in less harvestable yield reported. This can be abused by a malicious user in the following way:

- `poolReserves` = 1000
- `depositedInVault` = 1000
- vault generates 500 yield over a period of time
- Alice sees this and performs a Uniswap buy for 500.
- `poolReserves` = 1500
- `depositedInVault` = 1000
- `assetsWithYield` = 1500

When yield is calculated, the result will be $1500 - 1500 = 0$. Alice would then be able to enter the vault and earn the past yield at the expense of the real stakers.

RECOMMENDATION

Consider performing a sweep at the beginning of `BStaking._sync()`. This way any outstanding reserves will be deposited to the vault and the expression will increase bilaterally.

H-07 Yield can be manipulated

CONTINUED

PROOF OF CONCEPT <https://gist.github.com/ZdravkoHr/199a04d77171ae824106a0d951ad2a29>**— RESOLUTION**

Status: Resolved.

<https://github.com/0xBaseline/mercury/blob/ec7116192eec14270a5a5bd108f30f305f0a7d86/src/components/BStaking.sol#L196>

HIGH **H-08**

RESOLVED

Vault Rounding Perturbs Accounting

CATEGORY	Rounding
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

In the BSwap component, when a vault is attached to a pool the resulting value of the vault shares is not considered when updating the pool's reserves or the fee distribution amounts.

This creates a scenario where the Mercury system thinks it has received X assets to allocate in it's accounting in the totalBTokens and totalReserves variables, when at the end of the vault deposit action it only has e.g. x - 100 value of assets to allocate.

In particular this causes reverts when performing a sell or deleverage of any size after an `emergencyExit` or vault update has occurred, because the rounding error is realized, almost always making the `pool.reserves` less than the snapshot reserves, which leads to an underflow revert in the `getSafePriceBid` function because the `_params.poolReserves` is less than the `_maker.snapshotReserves`.

This can also cause the BLV invariant to be effectively breached when the system thinks that it holds more reserves than it's backing vault shares are actually worth.

The first attached POC shows that precision loss on the order of hundreds of wei is easily possible.

The second attached POC shows the DoS that occurs when this rounding is realized by the system.

— RECOMMENDATION

Instead of assuming that the provided user tokens value is what the system receives, for pools that have an attached vault consider doing the deposit up-front or previewing the results of the deposit and performing the swap accounting based on the `min(userProvidedReserves, vaultSharesValueInReserves)`. This way the Mercury system will not over-allocate value which it ultimately does not receive after a vault deposit due to rounding precision loss.

H-08 Vault Rounding Perturbs Accounting

CONTINUED

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-baselineamm-fuzz/blob/c90bf4488ee54275b34790929f97c941b926a21c/test/fuzzing/FoundryPlayground.sol#L14>

— RESOLUTION

Status: Resolved.

<https://github.com/0xBaseline/mercury/blob/7729a3ee5c7a6e7e8c1e11c00d1da160b4d68864/src/components/BSwap.sol#L307>

HIGH **H-09**

RESOLVED

Swap reentrancy enables yield manipulation

CATEGORY Reentrancy
LOCATION VaultLib.sol#L91-93
REVIEW Main Review

DESCRIPTION

`VaultLib.takeReserves()` first calls `NativeLib.handleIncoming()` and after that - `depositToVault()`. There is a native token refund logic in `handleIncoming` that allows the `msg.sender` to reenter the system before the assets are deposited to the vault.

A user can send a small surplus of native token to `BSwap.buyTokens()`, hijack the execution flow and reenter the `BStake` component to manipulate the yield.

`VaultLib.getHarvestableYield()` subtracts the holdings from the `assetsWithYield` to calculate the harvestable yield.

```
uint256 assetsWithYield = getAllocatedReserves(pool);  
uint256 holdings = pool.totalReserves - State.credit(_bToken).totalDebt +  
FeeLib.getUnclaimedFees(_bToken);  
return int256(assetsWithYield) - int256(holdings) - 1;
```

The `assetsWithYield` value stores the notional deposited value to the vault plus any yield earned. Since `takeReserves()` is the last step executed during swaps, when the user reenters, `pool.totalReserves` will have increased, but `assetsWithYield` will stay unchanged because `depositToVault` is not executed yet.

This can be abused by a malicious user in the following way:

- `poolReserves` = 1000
- `depositedInVault` = 1000
- vault generates 500 yield over a period of time
- Alice sees this and calls `buyTokens()`, spending 500 reserves, but sending 500 + 1 wei
- She inserts her own stake, which causes `_sync()` to be called
- `poolReserves` = 1500
- `depositedInVault` = 1000
- `assetsWithYield` = 1500

When yield is calculated, the result will be `1500 - 1500 = 0`. Alice will now earn part of the previous yield at the expense of the legitimate stakers.

H-09 Swap reentrancy enables yield manipulation

CONTINUED

— RECOMMENDATION

Consider reworking the refunding mechanism to execute the funds transfer at the very end of each flow. You can implement a flash accounting feature like UniswapV4 and clear it at the end of swaps, etc...

PROOF OF CONCEPT <https://gist.github.com/ZdravkoHr/df0cffbac91470143657d8e24e90a33e>

— RESOLUTION

Status: Resolved.

HIGH **H-10**

ACKNOWLEDGED

Leverage call is vulnerable to MEV sandwiches

CATEGORY MEV
LOCATION BCredit.sol#L343
REVIEW Main Review

DESCRIPTION

The `BCredit._leverage()` function calculates `borrowed` amount from `_totalCollateral` and then buys the difference between `_totalCollateral` and `_stakeToUse` from the pool.

```
(uint256 totalCost, ) = BSwap(address(this)).buyTokens(  
    _bToken,  
    _targetCollateral - _stakeToUse,  
    borrowed + _maxReservesIn  
);
```

The last argument passed to `buyTokens`, the `maxReservesIn` tells the pool how many tokens are we willing to pay. It's hardcoded to `borrowed + maxReserveIn`.

This is unnecessary high for small leverages. For example if `blvPrice = 1` and user calls `leverage()` with `_totalCollateral = 21k` and `_stakeToUse = 20k`, the system would be willing to use up `21k` reserves to buy only `1k` bTokens, which creates a MEV opportunity at the expense of the user.

RECOMMENDATION

Consider leaving only the `_maxReservesIn` parameter as a slippage protection.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-baselineamm-fuzz/blob/55da8f0e54ff10f1e68e68049d894287fe332427/test/fuzzing/FoundryPlayground.sol#L225-L296>

RESOLUTION

Status: Acknowledged.

HIGH **H-11**

RESOLVED

Inflation caused by claimed credits

CATEGORY	Logical Error
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

When a pool is created via `BFactory.createPool()`, the `initialCollateral` and `initialDebt` are added to `pool.totalBTokens` and `pool.totalReserves`.

Later, the credit is claimed by the users and the `bTokens` are being deposited to their staking account. However, `pool.totalBTokens` stays unchanged, even though the tokens should technically not be counted as available liquidity. The problem arises when users start repaying their debts and start withdrawing the staked `bTokens`. The `pool.totalBTokens` variable will not account for this, which will lead to wrong pricing of the tokens. For example, when users buy tokens, the following formula determines how much reserves they should pay.

$$\Delta y = P_t \times \Delta x \times (L_0/L_1)$$

The term `L0/L1` should adjust the price based on what part of the available supply is being traded. Because `pool.totalBTokens` includes assets not available in the pool, users would be able to buy tokens at a cheaper price.

Also, `_getCirculating()` will return a deflated value, which results in wrong results for all calculations depending on it, including `bookValue`.

This can also let users utilize the whole pool, because the system think it still holds the claimed tokens, and break the asymptote of `Xmin`.

RECOMMENDATION

Best solution would be to rework the claim system in a way that it's decoupled from the pool tokens.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-team1-1760986550486/blob/credit-inflates-poc/test/CreditInflatesPOC.t.sol>

H-11 Inflation caused by claimed credits

CONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/0xBaseline/mercury/blob/fb5e833193bac8e13703a0e8a4ba0f71758c90d0/src/components/BFactory.sol#L187>

HIGH **H-12**

RESOLVED

Slippage Affected By Circulating Supply

CATEGORY Gaming
 LOCATION Global
 REVIEW Main Review

DESCRIPTION

In the formula for the bid side liquidity curve, the circulating supply plays a large role in determining the `bufferConvexity` and thus the resulting slippage on trade execution.

This is however unrelated to the convexity of the ask side curve and is arbitrary based upon the total supply of the token which becomes a BToken. For any token that is made into a BToken from a pre-existing token this will be a significant issue.

For any token with a large totalSupply and large circulating supply, the slippage approaches zero on the bid curve, allowing for trivial buy then sell arbs which are not prevented by the arb guard.

For the execution result on sells we have figure 1:

Where B, the `bufferConvexity` is a result of figure 2:

As circulating grows, B becomes increasingly smaller, meaning the denominator in the overall execution equation approaches one. This trends towards an execution of simply the active price, and trends towards zero slippage.

PoC demonstration:

```
User starts with 942696430814026941867726 (9.42) Reserves & 0 BTokens
0. Buy 773190501915385216937475 (7.7e23) reserves for 413054796 (4.13e8 BTokens)
1. Sell 108677438 (1e8) BTokens for 203430727519948932490009 (2.03e23) Reserves
2. Buy 4233671906931141026299 (4e21) reserves for 2235659 (2.23e6) BTokens
3. Sell 40 bTokens for 75715769641563155 (7.5e16) reserves
4. Buy 208317289973047920252490 (2.08e23) for 85869309 (8.58e7) BTokens
The issue appears very clearly in the 4th step in this series of actions, but happens even in
smaller magnitudes in previous steps.
3. Sell 40 BTokens for 7.5e16 reserves
Execution price = 75715769641563155 / 40 = 1892894241039078 (1.89e15) Reserve / BToken
We simulate the user selling their entire BToken balance back to reserves, their simulated
total reserve balance is: 951482973394659285126224 (9.51 Reserves)
```

H-12 Slippage Affected By Circulating Supply

CONTINUED

```

4. Buy 15e7 BTokens for 4.36e23 reserves
Execution price = 436247359521852589227386 / 150017250 = 2907981312294770 (2.9e15) Reserve /
BToken
User BToken Balance 255926579 (25.5e7)
We simulate the user selling their entire BToken balance back to reserves, their simulated
total reserve balance is: 744229277605024711208198 (7.4e23) reserves
Simulated execution = 744229277605024711208198 / 255926579 = 2907979626473359 (2.9e15) Reserve
/ BToken
Notice! That the execution of this simulated (quoted) sell of the user's 25e7 BTokens produces
nearly zero slippage relative to the execution price of step 4. This means the arb guard
cannot protect against this action with the way that the arb guard is currently being applied
(sets the starting price, does not control net execution price).
So we push the price up with action 4, and then the simulated sell has _no_ slippage, so it's
always an immediate arb.
To understand the reason there is _no_ slippage on the bid side curve, let's have a look in
the calculation of the quote result.
Inside quoteReservesForTokensIn:
console::log("_params.activePrice", 291673390191713469080108 [2.916e23])
console::log("tokensIn", 255926579000000000000 [2.559e20])
console::log("BLV Price: ", 5080225833270765 [5.08e15])
console::log("bookPrice", 537935974247305203 [5.379e17])
console::log("curvePremium: ", 291672976511886445648115 [2.916e23])
console::log("bufferCoefficient", 5473767584 [5.473e9])
console::log("Reserves out: ", 74646868365599268927602776 [7.464e25])
We have the formula figure 1 above which represents the quote result.
The key here as we can see is that the bufferCoefficient is very small:
B = 5473767584
This is a 1e18 decimal thing, the reason it's so small is because the circulating supply is
large in this example.
console::log("_params.totalSupply", 10000000001129474278000000000000 [1e32]) (In the
trillions of BTokens)
console::log("_params.poolTokens", 450008156000000000000 [4.5e20])
Notice that BTokens have 6 decimals so these are scaled amounts.
Examine the bufferCoefficient calculation:
P - Pblv = 291673390191713469080108 - 5080225833270765 = 291673385111487635809343
Pbook - Pblv = 537935974247305203 - 5080225833270765 = 532855748414034438
=>
bufferCoefficient = ((291673385111487635809343 * 1e18 / 532855748414034438) - 1e18) /
10000000001129474278000000000000

```

H-12 Slippage Affected By Circulating Supply

CONTINUED

```
= (547377758388850847567689 - 1e18) * 1e18 / 10000000001129474278000000000000 = 5473767584  
(round up)
```

This result is small because the totalSupply is simply very large. This is an extreme example which shows a very large arbitrage percentage wise that demonstrates this behavior in the AMM. Such a scenario as shown explicitly in this PoC is reflective of any BToken that previously existed outside of the AMM before being added, if a significant portion of it's totalSupply will start outside the AMM.

However even on typical BTokens created within the protocol, a more realistic example can be crafted which still uses the fact that the Arb guard cannot protect against the overall execution of the sell here.

— RECOMMENDATION

The idea of the sell side execution formula is that the execution approaches the book price as the entire circulating supply is sold. The convexity with which it does so however needs to be much more steep to account for the fact that the slippage on the ask side can often be much less than the bid side when circulating supply is large.

However in having a higher slippage on the bid side and lower slippage on the ask side this would introduce a sell-then-buy-arb.

There is a fundamental fact that with two shifting curves that have disagreeing convexities there will always be arbitrage (not MEV) opportunities which take reserves from the pool, reducing book value.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-baselineamm-fuzz/blob/07f51a565a8da427faf16aced2dde79103c0f63a/test/fuzzing/FoundryPlayground.sol#L14>

— RESOLUTION

Status: Resolved.

<https://github.com/0xBaseline/mercury/pull/117/files>

HIGH **H-13**

ACKNOWLEDGED

Arb Guard Does Not Effectively Protect Execution

CATEGORY Gaming
LOCATION BSwap.sol
REVIEW Main Review

— DESCRIPTION

The Arb guard attempts to protect against trade arbs by assigning the starting price for a trade, however this is ultimately ineffective at restricting arbs since it is not the starting price of a trade which makes an arb, it is the overall execution.

When one curve has notably less slippage than the other, for example as described in H-13, then the arb guard ineffectively prevents the arbitrage from occurring. Since a much larger volume of tokens could be sold with very little execution slippage relative to the "safe price".

This allows for arbitrages of a high magnitude to still occur in the system.

— RECOMMENDATION

Some combination of solving the convexity differences between the two underlying curves and re-architecting the arb guard's method of restriction can address this. However the solution may entirely perturb the goal of the AMM in the first place and make for a significantly sub-optimal trading experience.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-baselineamm-fuzz/blob/07f51a565a8da427faf16aced2dde79103c0f63a/test/fuzzing/FoundryPlayground.sol#L14>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Unused Benchmark Offset

CATEGORY Logical Error
LOCATION BSwap.sol
REVIEW Main Review

DESCRIPTION

In the `_updateAsk` function the `maker.offsetSupply` is updated based on the `inventoryOffset` and `pnlOffset`. The `offsetSupply` is what defines the slippage that will be applied on the ask side, however it does not consider the `benchmarkOffset` even though it is calculated.

As a result the `benchmarkOffset` does not affect the `offsetSupply` result and therefore the benchmark tracking does not affect the Swap execution.

RECOMMENDATION

Include the `benchmarkOffset` as a contributor to the `offsetSupply` or remove it entirely if it is not needed.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Incorrect PnL offset for short positions

CATEGORY	Logical Error
LOCATION	BSwap.sol#L688
REVIEW	Main Review

DESCRIPTION

`BSwap._getPnlOffsetModifier()` uses the current realized `PnL` to compute a `breakevenPrice`. This is the price that would make `PnL` go to 0 if it was the true price. The function implements the following early return

```
if (_params.activePrice > uint256(breakevenPrice)) return 0;
```

While this may work for long positions, where negative PnL will result in higher `breakevenPrice` compared to the entry price, it fails for losing shorts. For example:

- size = -1
- entryPrice = 100
- pnl = -20

Here, `breakevenPrice` will be calculated as 80, since then the system would make +20 pnl and it would be zeroed out. However, any `activePrice` higher than `80` would cause the function to return early.

In addition to that, the `priceRatio` will always default to the second case.

```
uint256 priceRatio = _params.activePrice > uint256(breakevenPrice)
? _params.activePrice.divWad(uint256(breakevenPrice))
: uint256(breakevenPrice).divWad(_params.activePrice);
```

RECOMMENDATION

Invert the early return statement for short positions.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

Vault interactions don't check its limits

CATEGORY Compatibility
LOCATION Global
REVIEW Main Review

— DESCRIPTION

Pools in the system can be associated with a vault where the pool assets are deposited to and withdrawn from. However, the standard ERC4626 functions `maxDeposit()` and `maxWithdraw()` are not checked.

In the event of the protocol trying to deposit or withdraw more than possible, the transaction will be reverted. Both sweeping and transferring assets depend on the deposit logic which creates a risk of DOS-ing important user flows.

— RECOMMENDATION

Consider implementing a fallback for not executing deposits if `maxDeposit()` doesn't allow it. In that case, you will also have to account for that sum when calculating the harvestable yield and introduce a way to deposit it to the vault at a later point in time.

If `maxWithdraw()` doesn't allow withdrawals, assets will be trapped so there is nothing that can be done except waiting for the restriction to be removed. Note that in this case exiting the vault may also not be possible. You can consider pausing the system in such cases.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

RESOLVED

Pool creator can steal reserves from the pool

CATEGORY Reentrancy
LOCATION BController.sol#L147-152
REVIEW Main Review

DESCRIPTION

`BController.claimPoolFees()` first resets the creator and protocol fee variables to 0 and then executes `giveReserves()` for each of them in order to pay the funds to the recipients.

```
// Reset the creator claimable
pool.creatorClaimable = 0;
pool.protocolClaimable = 0;
// Transfer the fees to creator and protocol
pool.giveReserves(pool.creator, creatorFees, _asNative);
pool.giveReserves(meta.protocolFeeRecipient, protocolFees, true);
```

Each of the `giveReserves` calls performs a transfer. If the pool reserve is the wrapped token for the specific chain and `isNative = true` was used, the recipient will be sent native tokens.

The creator of the pool can benefit from the fact that both `creatorClaimable` and `protocolClaimable` are reset to 0, but only `creatorClaimable` has been withdrawn from the vault. This will decrease the relative value of the `holdings` variable in `VaultLib.getHarvestableYield` compared to the `assetsWithYield`, and `getHarvestableYield()` will return inflated yield, which will be redistributed back to the creator and the rest of the fee recipients. The creator can repeat this as many times as possible until the `protocolClaimable` variable is of meaningful amount. Note that this attack increases the risk of insolvency and may hinder important components of the AMM system - like the arbitrage guard, because the fees are paid of the reserves, but `totalReserves` is not changed.

RECOMMENDATION

The simplest solution would be to always send the wrapped version to the creator. This way you entirely eliminate the reentrancy vector, unless the wrapped token has hooks implemented.

M-04 Pool creator can steal reserves from the pool

CONTINUED

Otherwise, you can withdraw the vault assets before the external call and redeposit them again after it.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-team1-1760986550486/blob/creator-stealing-funds-poc/test/unit/CreatorReentrancyPOC.t.sol>

— RESOLUTION

Status: Resolved.

<https://github.com/0xBaseline/mercury/blob/fb5e833193bac8e13703a0e8a4ba0f71758c90d0/src/components/BController.sol#L166>

MEDIUM **M-05**

RESOLVED

Yield miscalculation because of deLeverage ()

CATEGORY Logical Error
LOCATION BCredit.sol#L388-389
REVIEW Main Review

DESCRIPTION

During `deleverage`, `credit.totalDebt` is first decreased, then `BStaking.liquidate()` is called and only after that is the swap performed and the assets withdrawn from the vault.

The call to `liquidate()` will also execute `_sync()`. Because `totalCredit` was decreased, but `totalReserves` is still not touched, the reported yield will be deflated. This causes uneven yield distribution and can be used by a malicious users to artificially decrease the harvestable yield, enter the system and start earning from it.

RECOMMENDATION

Perform the collateral unlock and the liquidation at the very end of the public `deleverage()` function.

RESOLUTION

Status: Resolved.

<https://github.com/0xBaseline/mercury/blob/e86b2daa879de8193fb6372acb904212f5ab8bd6/src/components/BCredit.sol#L406>

MEDIUM **M-06**

ACKNOWLEDGED

Execution price doesn't account for fees on sell

CATEGORY Logical Error
LOCATION BSwap.sol
REVIEW Main Review

DESCRIPTION

`BSwap._updatePosition()` is ran after every swap to update the `PnL` tracking state. It calculates the execution price of the current trade by dividing the amount of reserves by the amount of `bTokens`.

```
uint256 executionPrice = resWad.divWad(tokWad);
```

Before a buy is executed, fee is applied to the reserves amount that has to be sent by the user. Then `resWad` will represent `_reservesIn - fee_`, which is the net amount of reserves that enters the pool.

However, when selling, the fee is applied after the swap. Then `_updatePosition()` is called with `userReservesOut_`. Execution price will be computed as if the pool paid `userReservesOut`, while the actual amount is `userReservesOut + fee`.

This will result in incorrect `PnL` calculated and therefore wrong `Xmin` applied to the pool tokens.

RECOMMENDATION

Account for the fee when selling in `_updatePosition()`.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

ACKNOWLEDGED

Bid inverse pricing asymmetry

CATEGORY Logical Error
LOCATION CurveLib.sol#L157-160
REVIEW Main Review

DESCRIPTION

The inverse function for the bid curve - `quoteTokensForReservesOut()` - uses flat price when the convexity of the curve is 0.

```
// Zero-convexity fallback when book price is at or below baseline
if (bookPrice <= _params.blvPrice) return _reservesOut.divWadUp(bookPrice);
// Zero-convexity fallback when active price is at or below book
if (_params.activePrice <= bookPrice) return _reservesOut.divWadUp(bookPrice);
```

The first case, when `bookPrice < blvPrice`, ensures tokens are sold at the lower `bookPrice`, because if `blv` is used, some users may not be able to sell their tokens, due to the pool not having enough `yBacking`. While this violates the property that `bTokens` can be sold for at least `blv` at any point in time, it's better to have that check to avoid insolvency.

The second case, when `_params.activePrice <= bookPrice`, uses the higher `bookPrice` which will result in better trade for the user.

In contrast, the non-inverse `quoteReservesForTokensIn()` function prices the `bTokens` at `activePrice` when `convexity == 0`. This allows users to extract more value from the pool:

- via `sellTokens()` compared to `sellWithReserves()` in case 1
- via `sellWithReserves()` compared to `sellTokens()` in case 2

Note that this issue appears only after selling is enabled when convexity drops to 0.

RECOMMENDATION

Consider implementing case 1 in both functions:

```
if (bookPrice <= _params.blvPrice) return _reservesOut.divWadUp(bookPrice);
```

And using `activePrice` instead of `bookPrice` in case 2

M-07 Bid inverse pricing asymmetry

CONTINUED

```
- if (_params.activePrice <= bookPrice) return _reservesOut.divWadUp(bookPrice);  
+ if (_params.activePrice <= bookPrice) return _reservesOut.divWadUp(_params.activePrice);
```

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-team1-1760986550486/blob/bid-inverse-asymmetry-poc/test/unit/TestBidInverse.t.sol>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-08**

ACKNOWLEDGED

Risk of insolvency due to debt

CATEGORY Unexpected Behavior
LOCATION Global
REVIEW Main Review

— DESCRIPTION

The debt taken from `BCredit` is intentionally not reduced from the pool reserves. Debt given is `blv *` `collateral`, but because it's still considered part of the pool, it will contribute for a bigger curve premium during sells. If a lot of sells happen or there is a lot of debt, the pool may become insolvent since the debt is not yet returned.

— RECOMMENDATION

Consider redesigning the credit system or at least have a global maximum amount of debt that can exist.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-baselineamm-fuzz/blob/no-reserves-poc/test/fuzzing/FoundryPlayground.sol>

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

Unnecessary Price Comparison

CATEGORY Gas Optimization
LOCATION BSwap.sol
REVIEW Main Review

DESCRIPTION

In the `_getPnlOffsetModifier` function when the `_params.activePrice > uint256(breakevenPrice)` case holds the function early returns a 0 value.

However the following ternary operator uses `_params.activePrice > uint256(breakevenPrice)` as the condition. This is unnecessary as the `true` case has already been handled directly above.

RECOMMENDATION

Assign the `priceRatio` as `uint256(breakevenPrice).divWad(_params.activePrice)` always instead of including the ternary operator.

RESOLUTION

Status: Resolved.

LOW **L-02**

RESOLVED

AdminTransferred event is emitted twice

CATEGORY Events
LOCATION Relay.sol#L81-86
REVIEW Main Review

DESCRIPTION

The `Relay.acceptAdmin()` function:

- makes sure `msg.sender == pendingAdmin`
- calls `_setAdmin(pendingAdmin)`
- emits `AdminTransferred(msg.sender)`

```
function acceptAdmin() external {
    require(msg.sender == pendingAdmin, Relay_OnlyPendingAdmin());
    // Unset current admin's executor privileges
    executorExpiry[admin] = 0;
    // Complete transfer of admin privileges to pending admin
    _setAdmin(pendingAdmin);
    // Finally, clear out pending admin
    pendingAdmin = address(0);
    emit AdminTransferred(msg.sender);
}
```

The internal `_setAdmin(_admin)` function also emits the `AdminTransferred(_admin)` event. Since `_admin == msg.sender`, the exact same event is emitted twice when ownership is accepted.

```
function _setAdmin(address _admin) internal {
    admin = _admin;
    executorExpiry[_admin] = type(uint256).max;
    emit AdminTransferred(_admin);
}
```

RECOMMENDATION

Remove the event emission from `acceptAdmin()`.

L-02 AdminTransferred event is emitted twiceCONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/0xBaseline/mercury/blob/ec7116192eec14270a5a5bd108f30f305f0a7d86/src/Relay.sol#L68>

LOW **L-03**

RESOLVED

Initiating ownership transfer should emit event

CATEGORY Events
LOCATION Relay.sol#L70-72
REVIEW Main Review

DESCRIPTION

The `Relay` contract implements two-step ownership transfer by using `transferAdmin()` and `acceptAdmin()`. Calling `transferAdmin()` is an important action since it sets the `pendingAdmin` to prepare the contract for its new owner. However, no event is emitted in that function even though it makes sense for a potential owner to listen for such event and call `acceptAdmin()` when it's emitted.

RECOMMENDATION

Emit an event when `transferAdmin()` is called.

PROOF OF CONCEPT <https://github.com/0xBaseline/mercury/blob/ec7116192eec14270a5a5bd108f30f305f0a7d86/src/Relay.sol#L73>

RESOLUTION

Status: Resolved.

<https://github.com/0xBaseline/mercury/blob/e86b2daa879de8193fb6372acb904212f5ab8bd6/src/Relay.sol#L73>

LOW **L-04**

RESOLVED

Component doesn't properly support ERC165

CATEGORY Compatibility
LOCATION Component.sol#L36
REVIEW Main Review

DESCRIPTION

The `Component` contract implements the `supportsInterface()` function from `ERC165`, but it returns `true` only for its own interface.

```
function supportsInterface(bytes4 _interfaceId) external pure virtual returns (bool) {  
    return type(Component).interfaceId == _interfaceId;  
}
```

This means external integrators will receive `false` when they check the contract for `ERC165` support.

RECOMMENDATION

Add the `IERC165` `interfaceId` to the supported interfaces.

PROOF OF CONCEPT <https://github.com/0xBaseline/mercury/blob/ec7116192eec14270a5abd108f30f305f0a7d86/src/Component.sol#L38>

RESOLUTION

Status: Resolved.

LOW **L-05**

RESOLVED

claimPoolFees Does Not Sweep

CATEGORY	Unexpected Behavior
LOCATION	BController.sol
REVIEW	Main Review

— DESCRIPTION

The `claimPoolFees` function does not sweep funds from the Uniswap pool manager before attempting to give reserves to the relevant recipient addresses. As a result the function can unexpectedly fail when balances are pending a sweep.

This may even unexpectedly cause the claiming to fail if a swap is executed through the Uniswap V4 router before the `claimPoolFees` transaction is recorded in a block.

— RECOMMENDATION

Consider invoking `SweepLib.sweep(_bToken)` at the beginning of the `claimPoolFees` function.

— RESOLUTION

Status: Resolved.

<https://github.com/0xBaseline/mercury/blob/ec7116192eec14270a5a5bd108f30f305f0a7d86/src/components/BController.sol#L167>

LOW **L-06**

RESOLVED

getBufferConvexity() is not accessible

CATEGORY	DoS
LOCATION	BLens.sol
REVIEW	Main Review

— DESCRIPTION

`BLens.getBufferConvexity()` is external function that computes the buffer convexity by reading storage variables, but its selector is not included in the array returned by the `ROUTES()` function. Because of this, any attempt to call it on the Relay will result in revert - `Relay_RouteNotFound(0xac556f9c)`.

— RECOMMENDATION

Add the function's selector to the `ROUTES()` return value.

— RESOLUTION

Status: Resolved.

LOW **L-07**

RESOLVED

Rounding Asymmetry Causes getMaxBorrow Revert

CATEGORY DoS
LOCATION BCredit.sol
REVIEW Main Review

— DESCRIPTION

In the `BCredit` module, the leverage flow is careful to round the debt amount up for the user, using `mulWadUp` to compute the `debtWad` value in the `getBorrowForCollateral` function.

However in the `getMaxBorrow` and `_previewBorrow` function the `debtWad` value is not rounded up, yet this is also correctly rounding in the protocol's favor.

This is because the `leverage` function computes a debt value that will be assigned to the user based on the `targetCollateral` provided, thus rounding that debt value up.

And the `_previewBorrow` function computes a newTotalDebt to maxDebt ratio, which determines how much of the account's total collateral ought to be used for the corresponding debt amount. So the ratio should be maximized, and using maxDebt in the denominator means that maxDebt should therefore be minimized.

While both of these methods in leverage and borrow round in the correct manner, they create a disagreement in what the maxLeverage of an account can be, differing by 1 wei.

The leverage function says that the max leverage can be 1 wei higher than the borrow function because leverage rounds the maxDebt up while the borrow function rounds it down.

The getMaxBorrow function uses the more conservative rounding method of the borrow function, and this ultimately results in a revert when an account has levered to the maximum borrowable as determined by the leverage flow. Because the maxDebt computed conservatively in the getMaxBorrow function is actually 1 wei less than the amount that the account has already borrowed through leverage.

— RECOMMENDATION

Consider adding the following logic to avoid underflow panic reverts in the `getMaxBorrow` function in this case:

L-07 Rounding Asymmetry Causes getMaxBorrow Revert

CONTINUED

```
uint256 availableDebt;  
if (account.debt == maxDebt + 1) {  
    availableDebt = 0;  
} else {  
    availableDebt = maxDebt - account.debt;  
}
```

— RESOLUTION

Status: Resolved.

<https://github.com/0xBaseline/mercury/blob/4b8028e4262d4bc1582498e68a916d97aa61670e/src/components/BCredit.sol#L535>

LOW **L-08**

RESOLVED

Pool can be misconfigured

CATEGORY Validation
LOCATION BFactory.sol#L181
REVIEW Main Review

DESCRIPTION

`BFactory.createPool()` allows specifying `initialCollateral` and `initialDebt` for claim which are being added towards `totalReserves` and `totalBTokens`. However, the credit state is updated only if `initialCollateral > 0`. This allows the creation of a pool with `initialDebt` only. Due to the credit state not being updated, the pool will be misconfigured since its creation because `totalReserves` will increase, but `credit.totalDebt` won't. The debt cannot be cleared because a merkle root is not set.

RECOMMENDATION

Either update the state when `initialCollateral > 0 || initialDebt > 0` or revert in case of `initialCollateral == 0 && initialDebt > 0`.

RESOLUTION

Status: Resolved.

<https://github.com/0xBaseline/mercury/blob/f9c437c5150a3562767b2b2cfcc2ee95474a9353/src/components/BFactory.sol#L156>

LOW **L-09**

ACKNOWLEDGED

Possible division by 0

CATEGORY Math
LOCATION BStaking.sol#L268
REVIEW Main Review

DESCRIPTION

The `gain` calculation in `BStaking.getAccumulator()` divides by `timeToAdapt`. There is a `max()` used and a comment saying if `timeToAdapt` is too low, it will round to 1.

```
uint256 gain = FixedPointMathLib.min(
  ((err * timeElapsed) / uint256(State.meta().timeToAdapt)).max(1), // round up 1 if the
  timeToAdapt is too small
  err
);
```

However, the `max` function is applied after the division is performed, which bounds the end result to 1, not the denominator. In result a configuration of `timeToAdapt = 0` will revert during computation of `gain`, which may be unexpected if the anticipated behavior is to apply the whole `err` as `gain`.

RECOMMENDATION

Apply the `max(1)` call to the denominator instead.

```
uint256 gain = FixedPointMathLib.min(
  - ((err * timeElapsed) / uint256(State.meta().timeToAdapt)).max(1), // round up 1 if the
  timeToAdapt is too small
  + ((err * timeElapsed) / uint256(State.meta().timeToAdapt).max(1)), // round up 1 if the
  timeToAdapt is too small
  err
);
```

RESOLUTION

Status: Acknowledged.

LOW **L-10**

RESOLVED

Leverage rounds in users favor

CATEGORY Rounding
LOCATION BCredit.sol#L507
REVIEW Main Review

DESCRIPTION

`BCredit.leverage()` calls `getBorrowForCollateral()` to calculate `borrowing`, or how much the user should be paid.

```
(uint256 borrowed, uint256 fee) = getBorrowForCollateral(_bToken, _targetCollateral);
```

`getBorrowForCollateral()` uses `mulWadUp` to multiply the user collateral by the `blv` price. This means `debtWad` will be rounded up and respectively `debt` and `borrowAmount` will be larger values as well.

```
uint256 debtWad = blv.mulWadUp(collateralWad);  
uint256 debt = NormalizeLib.denormalizeWad(debtWad, rDec); // ceil  
fee_ = debt.mulWadUp(feeRate);  
borrowAmount_ = debt - fee_;
```

In result, the system will give the user 1 wei more debt in some cases. While this sounds negligible, it may be an incentive no not return the debt, since performing a `sellToken()` may not yield better result.

RECOMMENDATION

Round `debtWad` down.

```
- uint256 debtWad = blv.mulWadUp(collateralWad); // ceil for obligation  
+ uint256 debtWad = blv.mulWad(collateralWad);
```

RESOLUTION

Status: Resolved.

<https://github.com/0xBaseline/mercury/blob/4b8028e4262d4bc1582498e68a916d97aa61670e/src/components/BCredit.sol#L535>

LOW **L-11**

RESOLVED

Bid inverse calculates some values in user favor

CATEGORY Rounding
 LOCATION CurveLib.sol#L184-198
 REVIEW Main Review

DESCRIPTION

CurveLib.quoteTokensForReservesOut() computes `tokensIn` as `numerator / denominator`.

```
uint256 sqrtTerm = sqrtWadUp(base.mulWad(base) + 4 * K.mulWadUp(_reservesOut));
// Numerator: (bufferCoefficient×Δy - P₀) + √(...)
uint256 numerator = bufferCoefficientDy >= _params.activePrice
? (bufferCoefficientDy - _params.activePrice + sqrtTerm)
: (sqrtTerm - (_params.activePrice - bufferCoefficientDy));
// Solution: Δx = numerator / (2K)
// Protocol-favorable: add a 1-ULP cushion to denominator so floor/ceil bias never overstates
tokensOut
uint256 denom_ = 2 * K;
unchecked { denom_ += 1; }
tokensIn_ = FixedPointMathLib.fullMulDivUp(WAD, numerator, denom_);
```

However, it rounds `base.mulWad(base)` down, which is a part of the numerator and then adds 1 wei towards the denominator. This is rounding in favor of the user and may make the AMM give more `bTokens` that it should.

In addition, `K` which is used in the denominator is rounded down, is equal to `blv * (active - book) / [(book - blv) * circulating]`. While the end result is rounded down, the denominator `(book - blv) * circulating` is computed with `mulWad`, which will round it down as well, raising the value of `K`.

```
uint256 K = FixedPointMathLib.fullMulDiv(
_params.blvPrice,
_params.activePrice - bookPrice,
(bookPrice - _params.blvPrice).mulWad(circulating) //@audit -> rounds down => K is higher
);
```

L-11 Bid inverse calculates some values in user favor

CONTINUED

RECOMMENDATION

Round `base * 2` up, remove the 1 wei addition towards the denominator and round up the denominator of `K`

RESOLUTION

Status: Resolved.

LOW **L-12**

RESOLVED

Wrong benchmarks due to dynamic total supply

CATEGORY Logical Error
LOCATION BSwap.sol#L554
REVIEW Main Review

DESCRIPTION

In the `BSwap._updateBenchmark()` function, the circulating benchmark tokens are calculated by using the current total supply and the pool tokens from the benchmark.

```
uint256 benchCirculating = _getCirculating(  
    _params.totalSupply,  
    _maker.benchmarkTokens  
);
```

The function assumes `totalSupply` will stay constant, but `BToken` has a public `burn()` function which can reduce it. On top of that, any additional added token via `setDeployer()` that has a `mint()` function will also have its total supply increasing.

In result, benchmarks will be recorded incorrectly.

RECOMMENDATION

Record the `totalSupply` in the benchmark as well and use it when needed.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Labels cause truncation of longer contract names

CATEGORY Warning
LOCATION Component.sol#L42
REVIEW Main Review

DESCRIPTION

The helper function `Component.toLabel()` takes a `string` parameter and converts it to a `bytes32` value. The first 32 bytes will be used, while the rest of it will be discarded.

```
function toLabel(string memory _typeName) internal pure returns (bytes32) {  
    return bytes32(bytes(_typeName));  
}
```

In result the `LABEL` for a component with longer name than 32 bytes won't properly represent that name. This also increases the risk of label collisions.

RECOMMENDATION

Make sure contracts inheriting from the `Component` are not using names longer than 32 bytes.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Native token is not supported

CATEGORY	Informational
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

A reserve can be paired together with a BToken to create a UniswapV4 pool. To use native token, the reserve should be set to `address(0)`. However, Baseline doesn't support this setting. For example, `BSwap.initialize()` calls `reserve.decimals()` which will revert if the token is `address(0)`.

RECOMMENDATION

Document that the project doesn't support native tokens.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-03**

ACKNOWLEDGED

Swap fails if hookData.length is in the range [1;32)

CATEGORY Unexpected Behavior
LOCATION BHook.sol#L170-173
REVIEW Main Review

DESCRIPTION

BHook._swap() decodes the `limit` out of the `hookData` specified if the variable is not empty.

```
if (_hookData.length > 0) {  
  // decode the limit from the hook data  
  limit = abi.decode(_hookData, (uint256));  
} else {  
  // if no limit was provided via hook data then set to the max, and handle outside of the hook  
  (in a router)  
  limit = isExactIn ? 0 : type(uint256).max;  
}
```

This will result in a failure if `hookData.length > 0 && hookData.length < 32`. While this protects the user if they pass a wrong limit value, it can be an unexpected behavior.

RECOMMENDATION

Consider documenting this.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

ACKNOWLEDGED

solveBlvForConvexity() is a misleading name

CATEGORY	Best Practices
LOCATION	CurveLib.sol#L310
REVIEW	Main Review

— DESCRIPTION

`CurveLib.solveBlvForConvexity()` takes a `_targetConvexity` parameter and returns a new `BLV` price. The names of the function and its parameter imply that it will find a `BLV` price resulting in the desired buffer convexity. In reality, it accepts a `bufferPremiumRatio` and solves `BLV` according to it, not `bufferConvexity`.

— RECOMMENDATION

Rename the function to `solveBlvForPremiumRatio()`

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-05

RESOLVED

Incorrect PnL comment

CATEGORY Best Practices

LOCATION BSwap.sol#L682

REVIEW Main Review

DESCRIPTION

The comment above the `pnlPerToken` calculation in `BSwap._getPnlOffsetModifier()` says the absolute value of the position size is used, but that's not true.

```
// Breakeven calculation: P_be = P_entry - (PnL / |position|)
int256 pnlPerToken = FixedPointMathLib.sDivWad(_maker.realizedPnL, _maker.positionSize);
int256 breakevenPrice = int256(_maker.entryPrice) - pnlPerToken;
```

RECOMMENDATION

Remove the modulus from the comment.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-06

RESOLVED

borrowWithNative() should be renamed

CATEGORY	Best Practices
LOCATION	BCredit.sol#L103
REVIEW	Main Review

DESCRIPTION

The name of the `BCredit.borrowWithNative()` function suggest users can use native tokens to borrow reserves, but what really happens is they perform a normal borrow and receive the reserve asset as a native token (if applicable).

RECOMMENDATION

Rename the function to `borrowNative()`.

RESOLUTION

Status: Resolved.

<https://github.com/0xBaseline/mercury/blob/ec7116192eec14270a5a5bd108f30f305f0a7d86/src/components/BCredit.sol#L104>

INFORMATIONAL **I-07**

ACKNOWLEDGED

sync() frequency impacts the error correction

CATEGORY Warning
LOCATION BStaking.sol
REVIEW Main Review

DESCRIPTION

The correction to be applied on each step when `sync()` is called is calculated as the absolute difference between `targetTps` and `decayedTps`.

`decayedTps` is calculated as:

```
uint256 decayedTps = tokensPerSecond_.mulWad(_decayFactorWad(timeElapsed, timeToDistribute));
```

This is the implementation of the exponential decay. The smaller the `timeElapsed`, the bigger `decayedTps`. Therefore, as `sync()` is called more frequently, `decayedTps` will be a larger value compared to if it was called less often. This impacts the error correction, or `gain`, in the following way:

- less of it is applied if `targetTps > decayedTps`
- more of it is applied if `decayedTps > targetTps`

RECOMMENDATION

Document that `sync()` frequency impacts how much of a gain is being applied to the `tokensPerSecond`

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-08**

ACKNOWLEDGED

Console logs should be removed

CATEGORY Best Practices
LOCATION CurveLib.sol#L452-477
REVIEW Main Review

DESCRIPTION

`CurveLib.getSafePriceAsk()` has numerous calls to `console2.log()`. These increase the gas cost of the transaction and increase the risk of a revert if the network the contract is deployed on doesn't support such logs.

RECOMMENDATION

Remove the logs before deployment.

RESOLUTION

Status: Acknowledged.

HIGH **H-01**

RESOLVED

Fees Perturb Harvestable Yield

CATEGORY	Logical Error
LOCATION	BCredit.sol
REVIEW	Remediation Review

DESCRIPTION

In the leverage function, there is no longer any transferring of BTokens or Reserves in or out in the implementation of the function. Users are expected to have already deposited the requisite BTokens into their BStaking balance, and their leveraged assets go directly into their collateral and debt in the BCredit account.

As a result, no funds go into or come out of the vault during the accounting for the leverage flow. This means at the end of the leverage flow, the increase in poolReserves and unclaimedFees are must be offset by the increase of the credit.totalDebt.

This is because the harvest logic calculates the vault reserve holdings of the protocol as follows:

```
/// @notice Get the amount of assets in the vault the protocol has earmarked
function getReserveHoldings(BToken _bToken) internal view returns (uint256) {
    State.Pool storage pool = State.pool(_bToken);
    return
        pool.totalReserves
        + FeeLib.getUnclaimedFees(_bToken)
        - pool.idleReserves
        - State.hook(_bToken).outstandingReserves
        - State.credit(_bToken).totalDebt;
}
```

The leverage function upholds the invariant of not changing the reserve holdings of the vault at the end of it's execution, this is clearly visible with the following example:

- BToken Price: \$1
- BLV: \$0.50
- Borrowing fee = 10%
- Leverage: totalCollateral = 10e18, collateralIn = 8e18
- Borrow 5e18 reserves for 10e18 collateral
- Pay 0.5e18 in fees

H-01 Fees Perturb Harvestable Yield

CONTINUED

- 4.5e18 virtually goes into a swap, only 2e18 is actually used in the swap, 2.5e18 is refunded debt
- The final debt increase is 2.5e18
- The poolReserve Increase is 2e18
- The unclaimed fees increase is 0.5e18
- $2e18 + 0.5e18 - 2.5e18 = 0$ net change in reserves as tracked by the getReserveHoldings function

This is correct, however the intermediate result which is used to track the vault holdings when the BStaking.deposit function is invoked right after distributing fees but right before incrementing the totalDebt is incorrect.

At the time of invoking BStaking.deposit the accounting looks like this:

- BToken Price: \$1
- BLV: \$0.50
- Borrowing fee = 10%
- Leverage: totalCollateral = 10e18, collateralIn = 8e18
- Borrow 5e18 reserves for 10e18 collateral
- Pay 0.5e18 in fees
- Unclaimed fees increases by 0.5e18
- totalDebt has not increased at all
- getReserveHoldings reports 0.5e18 more assets in the system than exist, thus perturbing the yield calculation that is synced in deposit

— RECOMMENDATION

Perform the `distributeFees` invocation after the collateral is deposited and locked, since the depositing of collateral triggers a sync that relies on the state of the `unclaimedFees` accounting to be in line with the `credit.totalDebt` accounting.

PROOF OF CONCEPT <https://github.com/GuardianOrg/mercury-team1-1760986550486/blob/fe0e90f4ac02472c25197ef22a64ecf41954760b/test/unit/VaultLib.t.sol#L284>

— RESOLUTION

Status: Resolved.

<https://github.com/0xBaseline/mercury/blob/987044f432e990a71e6f6d7c0c7c3335fbdbea53/src/components/BCredit.sol#L266>

LOW **L-01**

ACKNOWLEDGED

No Fee Refund

CATEGORY Warning
LOCATION BCredit.sol
REVIEW Remediation Review

— DESCRIPTION

In the leverage function when the amount of borrowed reserves are greater than the amount required to obtain the collateralFromSwap the debt for this delta is forgiven with the debtRefund. However this debtRefund does not also apply to the fee amount, so the user pays the entire fee amount that would have applied to borrowing the full amount.

— RECOMMENDATION

It may introduce more complexity than it's worth to try to account for this and refund a portion of the fee, especially with respect to the harvestable yield calculations. Instead, simply be aware of this behavior and consider if it is acceptable for the protocol.

— RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.